Authentication & Security

Backend Engineering — Module 11

Study Handnote • Code Examples • Practice Labs

JWT • OAuth2 • Session Auth • SSO • SQL Injection
XSS • CSRF • API Authentication Attacks & Prevention

Table of Contents

> **How to use this note.** Read a section, then immediately do its *Practice* item in Section 11. Security knowledge decays fast if you only read it. The single highest-value habit is: **for every feature you build, ask "who is allowed to do this, and how do I prove the caller is that person?"**

1. Foundations

1.1 Authentication vs Authorization

Term	Question it answers	Example
Authentication (AuthN)	Who are you?	Verifying email + password, an OTP, a signed token, a client certificate.
Authorization (AuthZ)	What are you allowed to do?	User 42 may read invoice 99 because they own it; only `admin` may delete users.
Accounting (Audit)	What did you do?	Immutable audit log: "user 42 deleted invoice 99 at 10:02 from IP x".

> **Rule to memorise:** Authentication happens **once per request/session** at the edge. Authorization must happen **at every single resource access**, in the backend, never in the UI alone. Hiding a button is not authorization.

Related concepts

- **Identity** — the subject (user, service, device). Stable ID, e.g. `user_id=42`.
- **Credential** — the secret proving identity (password, private key, token).
- **Factor** — something you *know* (password), *have* (phone, key), *are* (biometric). MFA = 2+ different categories.
- **Principal** — the authenticated entity attached to a request context.
- **Claim** — a statement about a principal (`role=admin` , `email_verified=true`).

1.2 Password storage (the thing juniors get wrong most)

Never store passwords reversibly. Never use fast hashes (MD5, SHA-1, SHA-256) for passwords — GPUs compute billions of them per second. Use a **slow, salted, memory-hard KDF**.

Algorithm	Verdict	Notes
Argon2id	Best	Winner of Password Hashing Competition. Memory-hard, resists GPU/ASIC. Default choice for new systems.
scrypt	Good	Memory-hard, well-supported.
bcrypt	Fine	Battle-tested, everywhere. Cost 12+. Caveat: silently truncates input at 72 bytes.
PBKDF2	Acceptable	Use only when FIPS compliance forces it. Needs very high iteration counts.
SHA-256 / MD5 / SHA-1	Never	Too fast. Even "salted SHA-256" is broken for passwords.

```
// Node.js - bcrypt
import bcrypt from 'bcrypt';

const COST = 12;                                  // ~250ms per hash on modern CPU
const hash = await bcrypt.hash(plainPassword, COST);   // salt is generated + embedded
// stored value: $2b$12$Q4b1...  ->  algo$cost$salt+digest

const ok = await bcrypt.compare(plainPassword, hash);  // constant-time internally

// Node.js - Argon2id (preferred)
import argon2 from 'argon2';
const h = await argon2.hash(pw, {
  type: argon2.argon2id,
  memoryCost: 19456,   // 19 MiB  (OWASP baseline)
  timeCost: 2,
  parallelism: 1,
});
const valid = await argon2.verify(h, pw);
```

```
# Python - passlib / argon2-cffi
from argon2 import PasswordHasher
ph = PasswordHasher(memory_cost=19456, time_cost=2, parallelism=1)
digest = ph.hash(plain)
try:
    ph.verify(digest, plain)
    if ph.check_needs_rehash(digest):      # params upgraded since? re-hash on login
        digest = ph.hash(plain)
except Exception:
    pass  # invalid password
```

Salt vs Pepper

- **Salt** — random per-user value, stored *with* the hash. Defeats rainbow tables and makes identical passwords hash differently. Modern libraries do this for you.
- **Pepper** — a single secret key stored *outside* the DB (env var / KMS / HSM), mixed in via HMAC before hashing. Defends against a DB-only leak. Optional but nice: `hash(HMAC-SHA256(pepper, password))`.

> **Login endpoint anti-patterns**
>
> - **User enumeration:** "No such email" vs "Wrong password" tells an attacker which accounts exist. Return one generic message: "*Invalid email or password.*"
> - **Timing leak:** if the user doesn't exist you return in 2ms; if they do, bcrypt burns 250ms. Fix: always run a dummy hash comparison against a fixed fake digest when the user is missing.
> - **No rate limit:** unlimited attempts = free brute force. See Section 9.
> - **Password in logs / URL query string:** credentials must be in the request *body* over HTTPS, and scrubbed from logs.

```
// Uniform-timing login
const user = await db.users.findByEmail(email);
const digest = user?.passwordHash ?? DUMMY_HASH;   // fixed valid bcrypt hash
const passwordOk = await bcrypt.compare(password, digest);
if (!user || !passwordOk) {
  await recordFailedAttempt(email, req.ip);
  return res.status(401).json({ error: 'Invalid email or password' });
}
```

1.3 The transport layer is part of auth

- **TLS everywhere**, including internal service-to-service traffic. A token sent over plain HTTP is a token given away.
- **HSTS**: `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` — stops SSL-strip downgrade.
- Cookies must be `Secure` so browsers never send them over HTTP.
- Verify certificates in your HTTP clients. Never ship `rejectUnauthorized: false` / `verify=False`.

1.4 A working threat model (5-minute version)

Before designing any auth feature, answer these on paper:

1. **What am I protecting?** (user PII, money movement, admin capability)
2. **Who might attack it?** (anonymous internet, a logged-in user attacking *another* user, a malicious insider, a compromised dependency)
3. **What can they reach?** (any endpoint that doesn't require auth, plus any endpoint whose object ID they can guess)
4. **What happens if my token/DB/secret leaks?** (blast radius → drives token TTL, rotation, least privilege)
5. **How would I detect it?** (logs, alerts on failed-login spikes, refresh-token reuse)

> **Core principles that recur in every section below: Least privilege** (grant the minimum, for the minimum time) • **Defense in depth** (never one control) • **Fail closed** (on error, deny) • **Never trust client input** (including headers, cookies, JWT payloads before verification) • **Secure by default** (deny-all routes, opt-in to public).

2. Session Authentication

The classic, stateful model: the server remembers who is logged in; the client only holds an opaque pointer (the session ID) in a cookie.

2.1 How it works

Key property: the cookie value is *opaque* — it carries no information. All state lives server-side, so the server can revoke a session instantly by deleting one row/key.

2.2 Cookie attributes — know every one

Attribute	Value to use	What it prevents / does
`HttpOnly`	always on	JavaScript cannot read the cookie via `document.cookie`. Massively reduces impact of XSS (attacker can still *act* as the user, but can't exfiltrate the session).
`Secure`	always on	Cookie only sent over HTTPS. Prevents network sniffing / SSL-strip theft.
`SameSite`	`Lax` (default) or `Strict`	Controls cross-site sending → primary CSRF defense. `Lax`: sent on top-level GET navigations only. `Strict`: never cross-site. `None` requires `Secure` and reopens CSRF.
`Path`	`/`	Scope limiting. Weak isolation — not a security boundary.
`Domain`	omit it	Omitting = host-only cookie (safest). Setting `Domain=.example.com` shares it with *all* subdomains, including a compromised one.
`Max-Age` / `Expires`	match session policy	Absolute client-side lifetime. Server-side expiry must exist too — never trust the cookie.
`__Host-` prefix	`__Host-sid`	Browser *enforces*: Secure + Path=/ + no Domain. Stops a subdomain from overwriting your session cookie (cookie tossing).

```
Set-Cookie: __Host-sid=9f2b...e1; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=1800
```

2.3 Implementation (Express + Redis)

```
import session from 'express-session';
import { RedisStore } from 'connect-redis';
import { createClient } from 'redis';

const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

app.set('trust proxy', 1);                // needed behind a load balancer for Secure cookies

app.use(session({
  name: '__Host-sid',
  store: new RedisStore({ client: redis, prefix: 'sess:', ttl: 1800 }),
  secret: process.env.SESSION_SECRET,     // 32+ random bytes, from a secret manager
  resave: false,                          // don't rewrite unchanged sessions
  saveUninitialized: false,               // don't create sessions for anonymous visitors
  rolling: true,                          // refresh idle timeout on activity
  cookie: {
    httpOnly: true,
```

```
    secure: true,
    sameSite: 'lax',
    path: '/',
    maxAge: 30 * 60 * 1000,                // 30 min idle timeout
  },
}));

// LOGIN
app.post('/login', async (req, res) => {
  const user = await verifyCredentials(req.body);
  if (!user) return res.status(401).json({ error: 'Invalid email or password' });

  req.session.regenerate((err) => {        // *** session fixation defense ***
    if (err) return res.status(500).end();
    req.session.userId    = user.id;
    req.session.createdAt = Date.now();    // for absolute timeout
    req.session.save(() => res.json({ ok: true }));
  });
});

// AUTH MIDDLEWARE
const ABSOLUTE_MS = 8 * 60 * 60 * 1000;    // 8 hours hard cap
function requireAuth(req, res, next) {
  if (!req.session?.userId) return res.status(401).json({ error: 'Unauthenticated' });
  if (Date.now() - req.session.createdAt > ABSOLUTE_MS) {
    return req.session.destroy(() => res.status(401).json({ error: 'Session expired' }));
  }
  req.user = { id: req.session.userId };
  next();
}

// LOGOUT
app.post('/logout', (req, res) => {
  req.session.destroy(() => {
    res.clearCookie('__Host-sid', { path: '/' });   // must clear server-side AND client-side
    res.json({ ok: true });
  });
});
```

2.4 Session attacks and defenses

Attack	How it works	Defense
Session hijacking	Attacker steals the session ID (XSS, network sniffing, logs, Referer leakage).	HttpOnly + Secure + TLS; never put the sid in a URL; scrub cookies from logs; short idle timeout.
Session fixation	Attacker sets a known sid in the victim's browser *before* login (e.g. via a link or a subdomain), then reuses it after the victim authenticates.	**Regenerate the session ID on every privilege change** (login, role elevation, password change). Reject client-supplied unknown sids.
Predictable session ID	Sequential or timestamp-based IDs let attackers guess valid sessions.	≥128 bits from a CSPRNG (`crypto.randomBytes` , `secrets.token_urlsafe`). Never `Math.random()` .
Session not invalidated	Logout only deletes the cookie; the sid still works. Or password reset leaves old sessions alive.	Destroy server-side. On password change, delete *all* sessions for that user.
Cookie tossing	A compromised subdomain writes a cookie that overrides yours.	`__Host-` prefix; don't set a wildcard `Domain` ; isolate untrusted user content onto a separate registrable domain.
CSRF	Cookies are sent automatically on cross-site requests.	SameSite + CSRF token. Full treatment in Section 8.

2.5 Timeouts you must implement

- **Idle timeout** — expires after N minutes of inactivity (15–30 min for sensitive apps, hours for low risk).
- **Absolute timeout** — hard cap regardless of activity (8–24 h). Limits the value of a stolen session.
- **Re-authentication** — force a password/MFA re-entry for high-risk actions (change email, add payout account, delete org), even inside a valid session.

2.6 When to choose sessions

Choose server-side sessions when: you have a browser-facing app; you need instant revocation; you're on a single domain or a small set of first-party domains; you want the simplest correct thing.
Reality check: "sessions don't scale" is mostly a myth. A Redis lookup is sub-millisecond, and you'd probably hit Redis for other reasons anyway. Statelessness matters when you have *many independent services* that must validate identity without a shared store — that's where JWT earns its place.

3. JWT (JSON Web Token)

A JWT is a **self-contained, signed set of claims**. Instead of looking identity up in a store, the server verifies a signature and trusts the payload. RFC 7519.

3.1 Anatomy

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiI0MiIsImV4cCI6MTcwMH0 . 4Nx8s1c_kQ...
<-------- HEADER (b64url) --------->   <----- PAYLOAD ----->   <-- SIGNATURE -->

HEADER    { "alg": "HS256", "typ": "JWT", "kid": "key-2024-06" }
PAYLOAD   { "iss": "https://api.myapp.com",   // issuer
            "sub": "42",                      // subject = user id
            "aud": "myapp-web",               // audience
            "exp": 1717300000,                // expiry (seconds since epoch)
            "iat": 1717299100,                // issued at
            "nbf": 1717299100,                // not before
            "jti": "b7c1...",                 // unique token id (for denylist)
            "role": "admin", "scope": "orders:read" }   // custom claims
SIGNATURE HMACSHA256( b64url(header) + "." + b64url(payload), secret )
```

Base64 is encoding, not encryption. Anyone holding the token can read the payload (paste it into jwt.io). **Never put secrets, passwords, card numbers, or sensitive PII in a JWT.** The signature guarantees *integrity* (not tampered) and *authenticity* (we issued it) — not confidentiality. If you need confidentiality, use JWE (encrypted) or just keep it server-side.

3.2 Signing algorithms

alg	Type	Key	Use when
HS256/384/512	Symmetric HMAC	One shared secret signs *and* verifies	Issuer and verifier are the same service / same trust boundary. Simple monolith.
RS256 / PS256	Asymmetric RSA	Private key signs, public key verifies	Many services (or third parties) must verify but must not be able to mint tokens. Standard for OIDC.
ES256	Asymmetric ECDSA	Same as RS256, smaller/faster	Modern default for asymmetric. Shorter signatures.
`none`	—	—	Never. Must be explicitly rejected.

Two classic JWT breaks — understand these:

1. `alg: none` **attack.** Attacker edits the header to `{"alg":"none"}`, strips the signature, and rewrites the payload to `role: admin`. A naive library that "honours the header" accepts it. **Fix:** always pass an explicit algorithm allowlist to `verify()`.
2. **Algorithm confusion (RS256 → HS256).** Server verifies with RS256 using a public key. Attacker changes `alg` to HS256 and signs the token using the *public key as the HMAC secret*. If the library picks the algorithm from the header, verification succeeds. **Fix:** same — pin the algorithm server-side.

3.3 Signing and verifying (Node)

```
import jwt from 'jsonwebtoken';

// ---- ISSUE ----
const accessToken = jwt.sign(
  { sub: String(user.id), role: user.role, scope: 'orders:read orders:write' },
  process.env.JWT_SECRET,                       // 32+ random bytes
  {
    algorithm: 'HS256',
    expiresIn:  '15m',                          // SHORT. This is the revocation strategy.
    issuer:    'https://api.myapp.com',
    audience:  'myapp-web',
    jwtid:     crypto.randomUUID(),
  }
);

// ---- VERIFY (middleware) ----
function requireJwt(req, res, next) {
  const h = req.headers.authorization || '';
  if (!h.startsWith('Bearer ')) return res.status(401).json({ error: 'Missing token' });
  try {
    const claims = jwt.verify(h.slice(7), process.env.JWT_SECRET, {
      algorithms: ['HS256'],                    // *** allowlist: blocks none + confusion ***
      issuer:    'https://api.myapp.com',       // *** must validate iss ***
```

```
      audience:  'myapp-web',                  // *** must validate aud ***
      clockTolerance: 5,                       // seconds of clock skew
    });
    req.user = { id: claims.sub, role: claims.role, scope: claims.scope.split(' ') };
    next();
  } catch (e) {
    // TokenExpiredError -> 401 so the client knows to refresh
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

```
// WRONG - every line here is a real-world bug
const claims = jwt.decode(token);                // decode() does NOT verify the signature!
const claims = jwt.verify(token, secret);        // no algorithms allowlist -> confusion attack
jwt.sign(payload, secret);                       // no expiry -> token valid forever
jwt.sign({ password: user.password }, secret);   // payload is readable by anyone
const secret = 'secret123';                      // guessable HMAC key -> offline brute force
```

3.4 Access + Refresh token pattern

```
  Client                                Auth Service              Refresh Store (DB)
    |-- POST /login ---------------------->|                            |
    |<-- access(15m, in memory)            |-- store hash(refresh),     |
    |    refresh(30d, HttpOnly cookie) ----|   family_id, used=false ---->|
    |                                      |                            |
    |-- GET /api/x  Bearer access -------->| verify signature only       |
    |                                      |                            |
    |   ... 15 min later: 401 expired ...  |                            |
    |-- POST /refresh (cookie) ----------->| look up hash               |
    |                                      | if used==true  -> REUSE!   |
    |                                      |    revoke whole family ----->|
    |<-- new access + NEW refresh ---------| mark old used=true         |
```

	Access token	Refresh token
Lifetime	5–15 minutes	Days to weeks (sliding)
Sent to	Every API call, `Authorization: Bearer`	Only the `/refresh` endpoint
Storage	JS memory (best) or HttpOnly cookie	**HttpOnly, Secure, SameSite=Strict cookie, Path=/refresh**
Server state	None — stateless verify	Stored *hashed* in DB so it can be revoked

Refresh token rotation + reuse detection — the pattern to remember. Every refresh issues a brand-new refresh token and invalidates the old one. If an already-used refresh token is presented again, that means it was stolen (two parties hold it) → **revoke the entire token family** and force re-login. This turns a silent theft into a detectable event.

3.5 Where does the frontend store the token?

Location	XSS risk	CSRF risk	Verdict
`localStorage`	High — any injected JS reads it	None	Common but weak. A single XSS = full token theft, exfiltrated to attacker's server.
JS memory (variable)	Medium — stolen only while page is open	None	Good for the access token. Lost on refresh → pair with a refresh cookie.
HttpOnly cookie	Low — JS can't read it	Yes — auto-sent	Best storage, but you **must** add SameSite + CSRF protection.

Recommended combo: access token in memory, refresh token in a `HttpOnly; Secure; SameSite=Strict; Path=/auth/refresh` cookie.

3.6 The revocation problem

A signed JWT is valid until it expires — the server has no way to "un-issue" it. If a user is banned or logs out, their access token still works. Mitigations:

1. **Short TTL** (5–15 min) — bounds the damage window. The primary answer.
2. **Denylist by `jti`** — keep revoked token IDs in Redis with TTL = remaining lifetime. Adds a lookup, but only for revoked ones.
3. **Token version claim** — store `tokenVersion` on the user row; include it in the JWT; bump it on logout-all / password change. Requires a user lookup (cache it).
4. **Revoke the refresh token** — guarantees the session dies within one access-token lifetime.

3.7 Key management & JWKS

- With RS256/ES256 the issuer publishes public keys at `/.well-known/jwks.json`; verifiers fetch and cache them.
- The header's `kid` selects which key to use → enables **zero-downtime key rotation** (publish new key, sign with it, retire old key after max token lifetime).
- Validate `kid` against your known key set. Never fetch a key from a URL supplied inside the token (`jku` / `x5u` injection).

```
import { createRemoteJWKSet, jwtVerify } from 'jose';
const JWKS = createRemoteJWKSet(new URL('https://issuer.example.com/.well-known/jwks.json'));

const { payload } = await jwtVerify(token, JWKS, {
  issuer:   'https://issuer.example.com',
  audience: 'my-api',
  algorithms: ['RS256'],
});
```

3.8 JWT security checklist

☐ Explicit `algorithms` allowlist on verify	☐ Reject `alg: none`
☐ Validate `exp`, `nbf`, `iss`, `aud`	☐ Short access-token TTL (≤15 min)
☐ Strong secret (≥256-bit) or asymmetric keys	☐ Secret from a secret manager, never in git
☐ No sensitive data in the payload	☐ Never trust `decode()` output
☐ Refresh rotation + reuse detection	☐ Revocation path exists (jti / tokenVersion)
☐ HTTPS only; token never in a URL	☐ Re-check authorization per request, not just identity

3.9 Sessions vs JWT — the honest comparison

Dimension	Session (stateful)	JWT (stateless)
Server storage	Required (Redis/DB)	None for access tokens
Revocation	Instant, trivial	Hard — needs TTL + denylist
Horizontal scale	Needs shared store	Any node can verify independently
Cross-domain / mobile / microservices	Awkward	Natural fit
Payload size on the wire	~32 bytes	Hundreds of bytes to a few KB
Data freshness	Always current	Stale until token expires (role change lags)
Failure mode when done wrong	Fixation / hijacking	Forged or non-revocable tokens

Very common production answer: sessions for the first-party browser app, JWT for service-to-service and mobile/third-party APIs. Mixing is normal, not a smell.

4. OAuth 2.0

> **OAuth2 is a delegated *authorization* framework, not an authentication protocol.** It answers "may this app act on the user's behalf, for these scopes?" — not "who is this user?" Using a raw OAuth2 access token to log someone in is the classic mistake; for login you need **OpenID Connect** (Section 5), which adds an identity layer on top.

4.1 The four roles

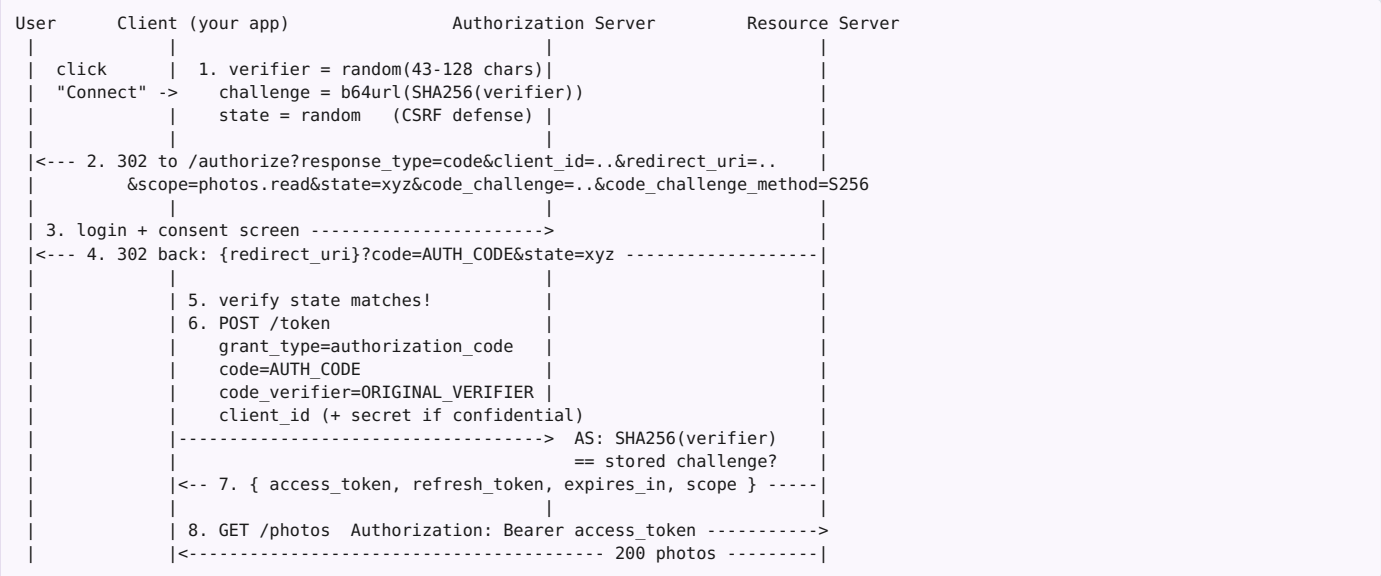
Role	Who	Example
Resource Owner	The user who owns the data	You
Client	The app requesting access	A photo-printing site
Authorization Server (AS)	Authenticates the user, issues tokens	accounts.google.com
Resource Server (RS)	The API holding the data	Google Photos API

Client types: Confidential (server-side app; can keep a `client_secret`) vs **Public** (SPA, mobile, desktop; cannot keep a secret → **must** use PKCE).

4.2 Grant types

Grant	Use for	Status
Authorization Code + PKCE	Web apps, SPAs, mobile — any user-present flow	Default choice
Client Credentials	Machine-to-machine; no user involved	Correct for service auth
Device Authorization	TVs, CLIs, input-constrained devices	Correct
Refresh Token	Getting new access tokens silently	Correct (rotate them)
~~Implicit~~	Was for SPAs; token returned in URL fragment	Deprecated — token leaks via history/Referer
~~Resource Owner Password (ROPC)~~	App collects the user's password directly	Deprecated — defeats the entire purpose

4.3 Authorization Code flow with PKCE, step by step



Why each piece exists

- **Code, not token, in the redirect.** The authorization code travels through the browser (URL → logs, history, Referer). It is single-use, short-lived (~60s), and useless without the client secret / code verifier.
- **PKCE** (Proof Key for Code Exchange, RFC 7636). On mobile/SPA, another app could register the same custom URL scheme and steal the code. PKCE binds the code to the client instance: only whoever knows the original `code_verifier` can redeem it. **Use PKCE even for confidential clients** — it's now the general recommendation.

- `state` — opaque random value echoed back; the client rejects mismatches. This is **CSRF protection for the OAuth callback**. Without it, an attacker can trick you into linking *their* account to *your* session.
- **Exact `redirect_uri` matching.** The AS must compare against a pre-registered URI byte-for-byte — no wildcards, no prefix matching. Otherwise an open redirect on your domain becomes a code-stealing chain.

```
// PKCE generation (browser / Node)
const verifier  = base64url(crypto.getRandomValues(new Uint8Array(32)));
const challenge = base64url(await crypto.subtle.digest('SHA-256', enc.encode(verifier)));
sessionStorage.setItem('pkce_verifier', verifier);   // keep verifier client-side, never send it in step 2

const url = new URL('https://as.example.com/authorize');
url.search = new URLSearchParams({
  response_type: 'code',
  client_id: CLIENT_ID,
  redirect_uri: 'https://app.example.com/callback',  // must match registration exactly
  scope: 'photos.read offline_access',
  state: randomState,
  code_challenge: challenge,
  code_challenge_method: 'S256',                     // never "plain"
});
location.assign(url);
```

```
// Server-side callback handler
app.get('/callback', async (req, res) => {
  const { code, state } = req.query;
  if (!state || state !== req.session.oauthState) return res.status(400).send('Bad state');
  delete req.session.oauthState;                     // single use

  const r = await fetch('https://as.example.com/token', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({
      grant_type: 'authorization_code',
      code,
      redirect_uri: 'https://app.example.com/callback',
      client_id: process.env.CLIENT_ID,
      client_secret: process.env.CLIENT_SECRET,      // confidential client only
      code_verifier: req.session.pkceVerifier,
    }),
  });
  const tokens = await r.json();                     // { access_token, refresh_token, expires_in }
  // store tokens SERVER-SIDE, keyed by the user's session. Never expose them to the browser.
});
```

4.4 Scopes and consent

- Scopes are coarse permission strings (`photos.read`, `orders:write`). Request the **minimum**.
- Scope is *not* a substitute for per-object authorization: a token with `orders:read` must still be checked against "is this order owned by this user?" (see BOLA, Section 9).
- `offline_access` is what gets you a refresh token on many providers.

4.5 Token validation at the Resource Server

Method	How	Trade-off
Local JWT validation	Token is a signed JWT; RS verifies with JWKS public key	Fast, no network call; but revocation lags
Token Introspection (RFC 7662)	RS posts the token to the AS `/introspect`, gets `{active: true, scope, sub, exp}`	Always current; network round trip → cache briefly

Also useful: **Revocation endpoint** (RFC 7009) to kill a token on logout, and `/.well-known/oauth-authorization-server` for metadata discovery.

4.6 OAuth2 attacks

Attack	Mechanism	Prevention
Missing `state` → CSRF	Attacker delivers their own auth code to your callback, linking their account to the victim's session (or vice versa).	Random `state` bound to the session, verified and single-use.
redirect_uri manipulation	Attacker registers/abuses a loose redirect (`https://app.com/*`, or an open redirect on your site) to receive the code.	Exact-match registered URIs; no wildcards; fix open redirects.

Authorization code interception	Malicious app on the same device claims the custom URL scheme and grabs the code.	**PKCE (S256)**; use claimed HTTPS app links.
Code replay	Same code redeemed twice.	Codes single-use and short-lived; AS revokes issued tokens if a code is reused.
Mix-up attack	Client supports multiple AS; attacker makes it send a code meant for AS-A to AS-B.	Per-AS distinct redirect URIs; validate the `iss` parameter in the response.
Token leakage	Token in URL → browser history, Referer header, server logs, analytics.	Never Implicit flow; tokens only in POST bodies / `Authorization` headers.
Overly broad scope	App asks for full account access to read a name.	Least privilege; users and reviewers should reject greedy scopes.
Refresh token theft	Long-lived credential stolen from storage.	Rotation + reuse detection; bind to client; store hashed.

5. SSO Basics

Single Sign-On: authenticate once with a central Identity Provider, then access many applications without logging in again. The apps never see the user's password.

5.1 Vocabulary

Term	Meaning
IdP (Identity Provider)	Holds the credentials and authenticates users. Okta, Azure AD/Entra, Google Workspace, Keycloak, Auth0.
SP (Service Provider) / RP (Relying Party)	Your application, which trusts the IdP's assertion of identity.
Assertion / ID token	The signed statement "this is user X, authenticated at time T, by method M".
Federation	A trust relationship between an SP and an IdP, established out-of-band (metadata exchange, client registration).
JIT provisioning	Creating the local user record on first SSO login from the assertion's attributes.
SCIM	Standard API for the IdP to push user create/update/deprovision to your app. Essential for enterprise offboarding.

5.2 The two protocols you'll meet

A. OpenID Connect (OIDC) — modern default

OIDC = **OAuth 2.0 + an identity layer**. Same Authorization Code + PKCE flow, plus:

- `scope=openid` triggers OIDC. Add `profile email` for basic attributes.
- `id_token` — a JWT *about the user*, returned alongside the access token. This is the authentication result.
- `nonce` — random value sent in the request and echoed inside the `id_token`; binds the token to your request (replay defense).
- **UserInfo endpoint** — call with the access token for additional claims.
- **Discovery** — `/.well-known/openid-configuration` lists all endpoints + JWKS URL.

```
id_token payload (decoded):
{
  "iss": "https://accounts.google.com",     // MUST equal the expected issuer
  "aud": "your-client-id.apps.google.com",  // MUST equal your client_id
  "sub": "108423...",                       // stable unique user id -> key your users on THIS
  "email": "amit@example.com",
  "email_verified": true,                   // check it before trusting the email!
  "nonce": "n-0S6_WzA2Mj",                  // MUST match what you sent
  "auth_time": 1717299000, "iat": ..., "exp": ...
}
```

ID token validation is mandatory. Verify: signature (via JWKS + `kid`), `iss`, `aud`, `exp` / `iat`, and `nonce`. Then: **key your local account on `iss` + `sub`, not on email.** Emails change and can be re-assigned; matching on an unverified email lets an attacker take over an account by registering that address at a sloppy IdP.

```
// OIDC login with the 'openid-client' library (Node)
import { Issuer, generators } from 'openid-client';

const google = await Issuer.discover('https://accounts.google.com');
const client = new google.Client({
  client_id: process.env.OIDC_CLIENT_ID,
  client_secret: process.env.OIDC_CLIENT_SECRET,
  redirect_uris: ['https://app.example.com/auth/callback'],
  response_types: ['code'],
});

// 1) redirect user
app.get('/auth/login', (req, res) => {
  const code_verifier = generators.codeVerifier();
  req.session.cv    = code_verifier;
  req.session.nonce = generators.nonce();
  req.session.state = generators.state();
  res.redirect(client.authorizationUrl({
    scope: 'openid email profile',
    code_challenge: generators.codeChallenge(code_verifier),
    code_challenge_method: 'S256',
```

```
    nonce: req.session.nonce,
    state: req.session.state,
  }));
});

// 2) handle callback
app.get('/auth/callback', async (req, res) => {
  const params = client.callbackParams(req);
  const tokenSet = await client.callback(
    'https://app.example.com/auth/callback',
    params,
    { code_verifier: req.session.cv, nonce: req.session.nonce, state: req.session.state }
  );                                  // library validates signature, iss, aud, exp, nonce
  const claims = tokenSet.claims();

  if (!claims.email_verified) return res.status(403).send('Email not verified by IdP');

  const user = await upsertUser({ idpIssuer: claims.iss, idpSubject: claims.sub,
                                  email: claims.email, name: claims.name });
  req.session.regenerate(() => {        // fresh session id after authentication
    req.session.userId = user.id;
    res.redirect('/dashboard');
  });
});
```

B. SAML 2.0 — the enterprise incumbent

- XML-based. The IdP returns a **SAML Assertion** (signed XML) posted to your **ACS** (Assertion Consumer Service) URL via an auto-submitting HTML form.
- **SP-initiated**: user hits your app → redirect to IdP → back with assertion. **IdP-initiated**: user starts at the IdP portal → assertion pushed to you (no request to correlate → weaker; prefer SP-initiated).
- Trust is configured by exchanging **metadata XML** (entity IDs, certificates, endpoints).

SAML validation musts: verify the XML signature against the IdP's registered certificate (and that the signature covers the assertion you actually read — **XML Signature Wrapping** attacks insert a second, unsigned assertion); check `Destination` matches your ACS URL; check `Audience` = your entity ID; enforce `NotBefore` / `NotOnOrAfter`; reject replayed `AssertionID` s; disable XML external entities (**XXE**). Use a mature library — never hand-roll SAML parsing.

5.3 OIDC vs SAML

Aspect	OIDC	SAML 2.0
Format	JSON / JWT	XML
Transport	REST + JSON, browser redirects	HTTP-Redirect / HTTP-POST binding
Best for	Web + mobile + APIs, consumer login	Legacy enterprise web apps, B2B tenants
Mobile support	Native	Poor
Complexity	Moderate	High (XML canonicalisation, signatures)
Also gives you	API authorization via access tokens	Attribute statements / group claims

5.4 Sessions in an SSO world

- There are **two sessions**: the IdP session (at the IdP) and the local app session (your cookie). They expire independently.
- **Single Logout (SLO)** is genuinely hard: OIDC offers RP-initiated logout, front-channel (hidden iframes) and back-channel (server-to-server logout token) logout. Back-channel is the most reliable; many teams settle for short local sessions instead.
- **Deprovisioning:** when HR disables an employee, your app must learn about it — via SCIM, back-channel logout, or short sessions that force re-auth against the IdP.
- **Blast radius:** SSO centralises risk. Compromise of the IdP account = compromise of every app → enforce MFA/phishing-resistant factors at the IdP.

Practical rule for "Login with Google/GitHub": it is OIDC (or OAuth + provider userinfo). Always (1) use Authorization Code + PKCE, (2) validate the ID token fully, (3) require `email_verified`, (4) store `iss` + `sub` as the account key, (5) regenerate your local session after login, and (6) think about the account-linking case: what happens when an existing password account has the same email? Require proving control of the existing account before linking, or you've built an account-takeover feature.

6. SQL Injection

Root cause: untrusted input is concatenated into a query string, so data crosses the boundary and becomes *code*. Same root cause as command injection, LDAP injection, and template injection.

6.1 The canonical example

```
// VULNERABLE
const q = `SELECT * FROM users WHERE email = '${email}' AND password_hash = '${hash}'`;
db.query(q);
```

Supply `email = ' OR '1'='1' --` and the query becomes:

```
SELECT * FROM users WHERE email = '' OR '1'='1' -- ' AND password_hash = '...'
```

The `--` comments out the rest; the `OR '1'='1'` is always true → returns the first user (often the admin). Authentication bypassed without knowing any password.

6.2 Categories

Type	Idea	Signal the attacker uses
In-band: UNION	Append `UNION SELECT` to pull data from other tables into the visible result set.	Data appears directly in the page.
In-band: Error-based	Force a database error whose message contains data.	Verbose DB errors returned to the client.
Blind: Boolean	Inject a condition and observe whether the page changes.	Different response for true vs false → extract data one bit at a time.
Blind: Time-based	Inject a conditional delay (e.g. a sleep function).	Response latency reveals the answer — works even with no output at all.
Out-of-band	Make the DB open a DNS/HTTP connection carrying the data.	Used when the response channel is unusable.
Second-order	Malicious value is stored safely, then later concatenated into a different query by another feature (e.g. a report job).	Sneaky — the sink is far from the source.

6.3 What an attacker gets

Read any table (credentials, PII, payment data) • modify or delete rows • bypass authentication • escalate to file read/write or OS command execution if the DB user is over-privileged • pivot to internal networks. SQLi is consistently one of the highest-impact web vulnerabilities.

6.4 Prevention — ranked

1. Parameterized queries / prepared statements primary defense

The query structure is sent to the database *separately* from the values. Values can never be re-interpreted as SQL, no matter what characters they contain.

```
// Node - node-postgres
const { rows } = await pool.query(
  'SELECT id, email FROM users WHERE email = $1 AND active = $2',
  [email, true]
);

// Node - mysql2 (prepared)
const [rows] = await conn.execute(
  'SELECT id FROM users WHERE email = ? AND tenant_id = ?', [email, tenantId]
);

# Python - psycopg (note: pass params as the 2nd arg, do NOT use % or f-strings)
cur.execute("SELECT id FROM users WHERE email = %s AND active = %s", (email, True))

# Python - SQLAlchemy Core
stmt = select(User).where(User.email == email)          # safe, parameterized

// Java - JDBC
```

```
PreparedStatement ps = conn.prepareStatement(
    "SELECT id FROM users WHERE email = ? AND active = ?");
ps.setString(1, email);
ps.setBoolean(2, true);
ResultSet rs = ps.executeQuery();

// Go - database/sql
row := db.QueryRow("SELECT id FROM users WHERE email = $1", email)
```

```
# These LOOK parameterized but are NOT - classic traps
cur.execute("SELECT * FROM users WHERE email = '%s'" % email)      # Python string format
cur.execute(f"SELECT * FROM users WHERE id = {user_id}")           # f-string
db.query("SELECT * FROM t WHERE a = " + a);                        // concatenation
session.query(User).filter(text(f"name = '{name}'"))               // raw text() with interpolation
knex.raw(`SELECT * FROM users WHERE id = ${id}`)                   // raw builder escape hatch
```

2. Allowlist anything that can't be a parameter

Table names, column names, `ORDER BY` direction, and `LIMIT` in some drivers cannot be bound as parameters. Never interpolate user input there — map it through a fixed allowlist.

```
const SORTABLE = { created: 'created_at', price: 'unit_price', name: 'display_name' };
const DIRS     = { asc: 'ASC', desc: 'DESC' };

const col = SORTABLE[req.query.sort] ?? 'created_at';   // unknown input -> safe default
const dir = DIRS[String(req.query.dir).toLowerCase()] ?? 'DESC';
const limit = Math.min(parseInt(req.query.limit, 10) || 20, 100);   // clamp numerics

const sql = `SELECT id, display_name FROM products
             WHERE tenant_id = $1 ORDER BY ${col} ${dir} LIMIT $2`;  // col/dir come from OUR map
await pool.query(sql, [tenantId, limit]);
```

3. Least privilege on the database account

- The app user should have only `SELECT/INSERT/UPDATE/DELETE` on the tables it needs — no `DROP`, no `CREATE`, no superuser, no file-system functions.
- Separate read-only credentials for reporting/analytics paths.
- This does not prevent SQLi — it caps the damage.

4. Defense in depth

- **Input validation** — validate type, length, format, range (e.g. an ID must be a positive integer). A useful second layer, *never* the primary one; blocklisting quotes/keywords is bypassable.
- **ORMs** parameterize by default — but audit every `raw()`, `text()`, `literal()`, `whereRaw()`, and native-query call.
- **Stored procedures** are safe only if they don't build dynamic SQL internally (`EXEC(@sql)` reintroduces the bug).
- **Generic error pages** — never return DB error text to clients; log details server-side.
- **WAF** — buys time against automated scanners; not a fix.
- **Monitoring** — alert on query errors spiking, unusual `UNION`/comment patterns, long-running queries.

6.5 NoSQL injection (same root cause)

```
// Express + Mongo. Body: { "email": "a@b.com", "password": { "$ne": null } }
db.users.findOne({ email: req.body.email, password: req.body.password });
//  -> matches any user whose password is not null. Login bypassed.
```

```
// Fix: enforce types before they reach the query
if (typeof req.body.email !== 'string' || typeof req.body.password !== 'string')
  return res.status(400).json({ error: 'Invalid input' });
// plus: validate with zod/joi, and never store plaintext passwords - compare hashes
```

6.6 Detection & testing

- Grep your codebase for string concatenation near `query`, `execute`, `raw`.
- Static analysis (Semgrep, CodeQL) with taint-tracking rules for SQLi sinks.
- Manual probes on a system **you own or are authorised to test**: a single quote causing a 500, or a boolean condition changing the result set, indicates injection.
- Add a regression test: send `' OR 1=1 --` as a value and assert the endpoint returns 0 rows / 400, not the whole table.

7. XSS (Cross-Site Scripting)

Root cause: untrusted data is inserted into an HTML/JS/CSS/URL context without encoding for that context, so the browser executes it as code *in the victim's origin*.

7.1 The three types

Type	Where the payload lives	Example
Stored (persistent)	Saved in your database, served to every viewer	A comment, profile bio, or product review containing a script tag. Highest impact — hits all users, including admins.
Reflected	In the request, echoed back in the response	Search page prints "No results for <query>". Delivered via a crafted link in email/chat.
DOM-based	Never touches the server — client-side JS writes attacker-controlled data into a dangerous sink	`el.innerHTML = location.hash.slice(1)` . Invisible to server-side scanners and WAFs.

7.2 What an attacker can do with XSS

Their JS runs with your site's origin and the victim's privileges, so: steal non-HttpOnly cookies and `localStorage` tokens • make authenticated API calls as the victim (this defeats CSRF tokens, because the script can read them) • keylog the login form • rewrite the page (fake payment form) • escalate: XSS in an admin panel → full compromise.

> **Mental model:** XSS = attacker-controlled JavaScript executing in your origin. `HttpOnly` stops cookie *theft*, but not *abuse* — the script can still act as the user. So HttpOnly is a mitigation, not a fix. The fix is: never let untrusted data become code.

7.3 Context matters — there is no single "escape" function

Context	Example	Required handling
HTML body	`<div>DATA</div>`	HTML-entity encode `& < > " '`
HTML attribute	`<div title="DATA">`	Entity encode **and always quote** the attribute (unquoted attrs break out on a space)
Inside <script>	`var x = "DATA";`	JS string encoding, or better: don't. Emit JSON via `<script type="application/json">` and parse it
URL / href	`<a href="DATA">`	URL-encode **and** allowlist the scheme — block `javascript:` and `data:`
CSS	`style="width:DATA"`	Avoid entirely; CSS can exfiltrate data
Event handler attr	`onclick="DATA"`	Never insert data here. Use `addEventListener`.

7.4 Vulnerable vs fixed

```
// SERVER (Express + template) - VULNERABLE
app.get('/search', (req, res) => {
  res.send(`<h1>Results for ${req.query.q}</h1>`);        // raw interpolation
});

// CLIENT - VULNERABLE DOM XSS
document.getElementById('out').innerHTML = params.get('name');
eval(userInput);
new Function(userInput)();
element.setAttribute('href', userInput);                    // javascript: payload
location.href = userInput;                                  // open redirect + javascript:

// REACT - VULNERABLE
<div dangerouslySetInnerHTML={{ __html: comment.body }} />   // the name is a warning
<a href={user.website}>site</a>                              // javascript: URL still executes
```

```
// SERVER - use a template engine with contextual auto-escaping
res.render('search', { q: req.query.q });     // EJS <%= %>, Nunjucks/Jinja {{ }}, Handlebars {{ }}
// (avoid the "raw/unescaped" variants: <%- %>, {{{ }}}, |safe)

// CLIENT - textContent never parses HTML
document.getElementById('out').textContent = params.get('name');
```

```
// URL scheme allowlist
function safeUrl(u) {
  try {
    const url = new URL(u, location.origin);
    return ['http:', 'https:', 'mailto:'].includes(url.protocol) ? url.href : '#';
  } catch { return '#'; }
}

// REACT - JSX auto-escapes text by default; sanitize only when you must render HTML
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{
  __html: DOMPurify.sanitize(comment.body, { ALLOWED_TAGS: ['b','i','em','strong','a','p','ul','li'],
                                             ALLOWED_ATTR: ['href'] })
}} />
```

Dangerous sinks to grep for

`innerHTML` • `outerHTML` • `insertAdjacentHTML` • `document.write` • `eval` • `new Function` • `setTimeout/setInterval` with a string • `jQuery .html() / .append() / $(userInput)` • `dangerouslySetInnerHTML` (React) • `v-html` (Vue) • `bypassSecurityTrustHtml` (Angular) • `location` / `window.open` assignments.

7.5 Content Security Policy (CSP) — the safety net

CSP tells the browser which sources may execute. It doesn't fix the bug, but it can turn a working XSS into a blocked one.

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-r4nd0mPerRequest' 'strict-dynamic';
  object-src 'none';
  base-uri 'none';
  frame-ancestors 'none';
  form-action 'self';
  require-trusted-types-for 'script';
  report-uri /csp-report

<!-- only scripts carrying the matching per-request nonce run -->
<script nonce="r4nd0mPerRequest"> /* app code */ </script>
```

- **Nonce-based CSP** (fresh random per response) is the recommended modern form; long allowlists of CDNs are usually bypassable.
- `'unsafe-inline'` and `'unsafe-eval'` defeat most of the benefit — removing them is the actual work.
- `object-src 'none'` and `base-uri 'none'` close well-known bypasses.
- Roll out with `Content-Security-Policy-Report-Only` first and watch the reports.
- **Trusted Types** (`require-trusted-types-for 'script'`) structurally eliminates DOM XSS by making raw strings unassignable to sinks.

7.6 XSS defense checklist

☐ Contextual output encoding everywhere	☐ Use framework auto-escaping; audit every "raw" escape hatch
☐ `textContent` instead of `innerHTML`	☐ Sanitize rich text with DOMPurify (allowlist)
☐ URL scheme allowlist for `href` / `src`	☐ No `eval` / string `setTimeout`
☐ Strict nonce-based CSP	☐ `HttpOnly` cookies (limits token theft)
☐ Validate on input, encode on output	☐ Set `X-Content-Type-Options: nosniff` + correct `Content-Type`
☐ Serve user-uploaded files from a separate domain	☐ Escape data in JSON embedded in HTML (`</script>` breakout)

8. CSRF (Cross-Site Request Forgery)

Root cause: ambient authority. Browsers automatically attach cookies to requests for a site — *regardless of which site initiated the request*. So a page on `evil.com` can cause an authenticated request to `bank.com`. The attacker cannot read the response (blocked by the Same-Origin Policy) but the **state change already happened**.

8.1 The attack

```
1. Victim logs into bank.com  ->  Cookie: sid=abc (HttpOnly, Secure)
2. Victim, still logged in, visits evil.com
3. evil.com serves:

   <form action="https://bank.com/transfer" method="POST" id="f">
     <input type="hidden" name="to"     value="attacker-account">
     <input type="hidden" name="amount" value="10000">
   </form>
   <script>document.getElementById('f').submit();</script>

4. Browser POSTs to bank.com AND ATTACHES Cookie: sid=abc
5. bank.com sees a valid session -> transfers the money
6. Attacker never sees the response - and doesn't need to.
```

GET-based variants are even easier (`<img src="https://bank.com/delete?id=5">`) — which is why **state-changing operations must never be GET**. But note: using POST alone is *not* a defense, as the example above shows.

8.2 Defenses

1. SameSite cookies first line

Value	Behaviour	Notes
`Strict`	Never sent on any cross-site request	Strongest. Downside: following a link from an email/another site shows you logged out.
`Lax`	Sent only on top-level **GET** navigations	Modern browser default. Blocks cross-site POST/PUT/DELETE and sub-resource requests. Good balance.
`None`	Always sent; requires `Secure`	Needed for legitimate cross-site flows (embedded widgets, separate API domain) — you then **must** add token-based CSRF defense.

Why SameSite isn't sufficient alone: browser defaults vary and older clients ignore it; `Lax` still allows cross-site GET (so a state-changing GET stays exploitable); a compromised *subdomain* is same-site, not cross-site. Use it **with** a token.

2. Synchronizer token pattern classic, strongest

```
// Issue: random token bound to the session, delivered in the HTML (not readable cross-origin)
app.use((req, res, next) => {
  if (!req.session.csrf) req.session.csrf = crypto.randomBytes(32).toString('base64url');
  res.locals.csrfToken = req.session.csrf;
  next();
});

// Template:  <input type="hidden" name="_csrf" value="<%= csrfToken %>">
//   or SPA:  <meta name="csrf-token" content="...">  -> send as X-CSRF-Token header

// Verify on every state-changing method
function verifyCsrf(req, res, next) {
  if (['GET', 'HEAD', 'OPTIONS'].includes(req.method)) return next();   // safe methods
  const sent = req.body?._csrf || req.get('X-CSRF-Token') || '';
  const real = req.session.csrf || '';
  const a = Buffer.from(sent), b = Buffer.from(real);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b))           // constant-time compare
    return res.status(403).json({ error: 'CSRF token invalid' });
  next();
}
app.use(verifyCsrf);
```

Rotate the token on login/privilege change. Per-session tokens are fine; per-request tokens are stricter but break the back button and parallel tabs.

3. Double-submit cookie (stateless)

Send the same random value both as a cookie and as a header/field; the server checks they match. Works without server-side state, but a subdomain that can set cookies on your domain can forge it. Use the **signed** variant: the cookie value is HMAC'd

with a server key and bound to the session ID.

4. Origin / Referer validation

```
const ALLOWED = new Set(['https://app.example.com']);
function checkOrigin(req, res, next) {
  if (['GET','HEAD','OPTIONS'].includes(req.method)) return next();
  const origin = req.get('Origin') || (req.get('Referer') && new URL(req.get('Referer')).origin);
  if (!origin || !ALLOWED.has(origin)) return res.status(403).json({ error: 'Bad origin' });
  next();   // fail closed when the header is absent
}
```

5. Custom header + CORS preflight (for JSON APIs)

A cross-origin request carrying a custom header (e.g. `X-Requested-With`) or `Content-Type: application/json` triggers a **CORS preflight**, which the attacker's origin will fail. So: require JSON content type, require a custom header, and **reject** `application/x-www-form-urlencoded`, `multipart/form-data`, **and** `text/plain` on state-changing endpoints — those are the "simple" types that bypass preflight.

CORS misconfiguration turns into CSRF-plus-read. Never do this:

```
Access-Control-Allow-Origin: <reflected request Origin>
Access-Control-Allow-Credentials: true
```

Reflecting arbitrary origins with credentials lets any site read authenticated responses. Use a strict allowlist, and never pair `*` with credentials (browsers block it, but reflection sneaks past). Also validate the *whole* origin — `evil-example.com` matches a naive `endsWith('example.com')` check.

8.3 When are you NOT vulnerable to CSRF?

Auth mechanism	CSRF?	Why
Session cookie	Yes	Sent automatically by the browser
JWT in a cookie	Yes	It's still a cookie — the token type is irrelevant
HTTP Basic / Digest	Yes	Browser re-sends credentials automatically
JWT in `Authorization: Bearer` header	No	Browsers never attach it automatically; attacker's JS can't set it cross-origin

Login CSRF is real too: an attacker forces you to log into *their* account, then your subsequent activity (searches, saved cards) lands in it. Protect the login form with a CSRF token as well.

8.4 Extra hardening

- **Re-authenticate or step-up MFA** for high-value actions (transfers, email change, adding payout details). Defeats CSRF and stolen sessions alike.
- **Never use GET** for state changes; enforce method semantics.
- **Remember:** XSS defeats every CSRF defense — injected script can read the token. Fix XSS first.

9. API Authentication Attacks & Prevention

APIs fail differently from web pages: there's no UI to hide behind, endpoints are enumerable, and object IDs are guessable. The OWASP API Security Top 10 is dominated by **authorization** bugs, not cryptography.

9.1 Broken Object Level Authorization (BOLA / IDOR) #1 API risk

The caller is authenticated, but the server never checks whether *this* user may access *that* object. Change the ID in the URL → read someone else's data.

```
// VULNERABLE - authentication without authorization
app.get('/api/invoices/:id', requireAuth, async (req, res) => {
  const inv = await db.invoices.findById(req.params.id);   // whose invoice? nobody asked
  res.json(inv);
});
```

```
// FIXED - scope every query by the authenticated principal
app.get('/api/invoices/:id', requireAuth, async (req, res) => {
  const inv = await db.invoices.findOne({
    id: req.params.id,
    tenantId: req.user.tenantId,      // multi-tenant boundary
    ownerId: req.user.id,             // or: check a membership/ACL table
  });
  if (!inv) return res.status(404).end();   // 404, not 403 - don't confirm existence
  res.json(inv);
});
```

Structural fixes that scale better than remembering:

- Push the tenant/owner filter into the data layer (repository methods that *require* a principal; Postgres Row-Level Security).
- Use unguessable IDs (UUIDv4/ULID) — obscurity only, but it stops trivial enumeration.
- Centralise policy: a single `can(user, 'read', resource)` function, or a policy engine (Casbin, OPA, Oso).
- Write an automated test per endpoint: "user B requests user A's object → expect 404".

Broken Function Level Authorization

Same bug at the *operation* level: a normal user calls `DELETE /api/users/7` or `/api/admin/reports` and it works because the check lived only in the frontend. **Deny by default**: every route requires an explicit role/permission declaration; new routes are inaccessible until someone declares who may call them.

9.2 Attacks on the credentials themselves

Attack	How it works	Prevention
Credential stuffing	Replays username/password pairs leaked from other breaches. Works because people reuse passwords.	MFA; breached-password checks (k-anonymity API); per-account + per-IP rate limits; device fingerprint / impossible-travel alerts; CAPTCHA on anomaly.
Brute force / password spraying	Many passwords against one account, or one common password against many accounts (evades per-account lockout).	Rate limit by account *and* by IP/ASN; exponential backoff; strong password policy (length > complexity); monitor global failure rates.
Token / session theft	Token leaked via XSS, logs, URL, or a compromised device.	HttpOnly cookies; short TTLs; never log or URL-embed tokens; refresh rotation with reuse detection; bind tokens to client (DPoP / mTLS).
Replay attack	A captured valid request is re-sent (double charge, repeated transfer).	Signed requests with timestamp + nonce; idempotency keys; short token lifetime; TLS.
OTP / reset-token brute force	6-digit OTP = 1M options; unlimited tries makes it trivial.	Max 5 attempts then invalidate; OTP TTL ≤ 5 min; single use; ≥128-bit reset tokens stored hashed; invalidate on use and on password change.
Account enumeration	Different responses/timing for existing vs non-existing accounts on login, signup, or reset.	Identical messages, status codes, and timings across all three flows.
JWT forgery	`alg:none`, algorithm confusion, weak HMAC secret cracked offline.	See Section 3.8.
Mass assignment	`PATCH /users/me {"role":"admin","isVerified":true}` gets	Explicit allowlist of updatable fields (DTO / zod schema); never `Object.assign(user, req.body)`.

	bound straight onto the model.	
Excessive data exposure	Endpoint returns the whole row (password hash, internal flags, other users' emails) and the UI just hides it.	Serialise through explicit response DTOs; never `res.json(dbRow)`.
SSRF via auth callbacks	Attacker-supplied URL (webhook, avatar fetch, OIDC discovery) makes your server call internal metadata endpoints.	Allowlist hosts, block private IP ranges, resolve-then-validate, no redirects.

9.3 Rate limiting & lockout

```
// Layered limits: strict on auth endpoints, looser on general API
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';

const loginLimiter = rateLimit({
  store: new RedisStore({ sendCommand: (...a) => redis.sendCommand(a) }),
  windowMs: 15 * 60 * 1000,
  limit: 5,                                  // 5 attempts / 15 min
  keyGenerator: (req) => `${req.ip}:${req.body.email ?? ''}`,   // IP + account
  standardHeaders: true,                     // RateLimit-* response headers
  skipSuccessfulRequests: true,              // only count failures
  handler: (req, res) => res.status(429).json({ error: 'Too many attempts. Try again later.' }),
});
app.post('/login', loginLimiter, loginHandler);
```

- **Progressive delay** beats hard lockout: hard lockout is itself a denial-of-service vector against your users.
- Limit by **account**, **IP**, and **global** — each catches a different attack shape.
- Apply to: login, signup, password reset, OTP verify, token refresh, and any expensive endpoint.
- Behind a proxy, trust `X-Forwarded-For` only from your own load balancer.

9.4 Machine-to-machine authentication

Mechanism	How	When
API key	Long random string, sent in a header. Store only a **hash** server-side; show the plaintext once.	Simple partner access. Weak: static, no expiry by default → add scopes, rotation, and per-key rate limits.
OAuth2 Client Credentials	Service exchanges client_id/secret for a short-lived access token.	Standard for internal/partner service auth. Preferred over raw API keys.
HMAC request signing	Client signs method+path+body-hash+timestamp+nonce with a shared secret; server recomputes.	Webhooks, payment APIs. Gives integrity + replay protection, not just authentication.
mTLS	Both sides present X.509 certificates.	Zero-trust internal meshes, high-assurance partners. Strongest, most operational overhead.
Workload identity	Short-lived credentials issued by the platform (SPIFFE, cloud IAM roles).	Best modern practice — no long-lived secrets to leak.

```
// Verifying an HMAC-signed webhook - includes replay protection
import crypto from 'node:crypto';

function verifyWebhook(req, res, next) {
  const ts  = req.get('X-Timestamp');
  const sig = req.get('X-Signature');
  if (!ts || !sig) return res.status(401).end();

  // 1) reject stale requests (replay window)
  if (Math.abs(Date.now() / 1000 - Number(ts)) > 300) return res.status(401).end();

  // 2) recompute over timestamp + RAW body (use express.raw, not the parsed object)
  const expected = crypto.createHmac('sha256', process.env.WEBHOOK_SECRET)
                         .update(`${ts}.${req.rawBody}`)
                         .digest('hex');

  // 3) constant-time compare - '===' leaks timing information
  const a = Buffer.from(sig), b = Buffer.from(expected);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) return res.status(401).end();

  // 4) optional: store the nonce/event id in Redis with a TTL to block exact replays
  next();
}
```

9.5 Operational hygiene

- **Secrets management:** never in git, never in the image, never in frontend bundles. Use a manager (Vault, AWS/GCP Secrets Manager, Doppler) with rotation. Scan repos (gitleaks, trufflehog) and rotate anything that ever touched a commit — history is forever.
- **Logging:** log auth *events* (login success/failure, token refresh, permission denied, role change) with user id, IP, user agent, and timestamp. **Never log** passwords, tokens, cookies, OTPs, or full card numbers. Redact at the logger level, not at the call site.
- **Monitoring & alerting:** spikes in 401/403, failed logins per account, refresh-token reuse, logins from new countries, mass 404s on sequential IDs (enumeration in progress).
- **Dependency hygiene:** most breaches start with a known CVE. Automate `npm audit`/Dependabot/Snyk; pin and verify lockfiles.
- **Error handling:** generic messages to clients, detailed context to logs with a correlation ID.

9.6 Security headers reference

Header	Purpose
`Strict-Transport-Security: max-age=31536000; includeSubDomains`	Force HTTPS, stop downgrade
`Content-Security-Policy: ...`	Constrain script sources → XSS mitigation
`X-Content-Type-Options: nosniff`	Stop MIME sniffing turning uploads into scripts
`X-Frame-Options: DENY` / `frame-ancestors 'none'`	Clickjacking protection
`Referrer-Policy: strict-origin-when-cross-origin`	Stop URL (and token) leakage via Referer
`Permissions-Policy: geolocation=(), camera=()`	Disable unused browser features
`Cache-Control: no-store` on auth responses	Keep tokens out of shared caches / disk

In Express: `app.use(helmet())` sets sensible defaults — then tune CSP yourself.

10. Cheat Sheets

10.1 Which auth mechanism should I use?

Situation	Choose
Server-rendered app, one domain	Server-side **sessions** + SameSite cookies + CSRF token
SPA + your own API, same site	Sessions, or JWT in HttpOnly cookie + CSRF protection
Mobile app → your API	OAuth2 **Authorization Code + PKCE**, short access token + rotating refresh
Third-party app accessing user data	**OAuth2** Authorization Code + PKCE with scopes
"Log in with Google/Microsoft"	**OIDC** (validate the id_token properly)
Enterprise customer demands SSO	**SAML 2.0** or OIDC, plus SCIM for provisioning
Service → service, no user	**Client Credentials**, mTLS, or workload identity
Inbound webhooks	**HMAC signature** + timestamp + nonce

10.2 Vulnerability → root cause → primary fix

Vulnerability	Root cause	Primary fix
SQL Injection	Data concatenated into query syntax	Parameterized queries + allowlists for identifiers
XSS	Data inserted into HTML/JS without contextual encoding	Contextual output encoding + sanitizer + strict CSP
CSRF	Cookies attached automatically cross-site	SameSite + anti-CSRF token + Origin check
BOLA / IDOR	Authentication without per-object authorization	Scope every query by the principal; centralised policy
Session fixation	Session ID survives the privilege change	Regenerate session ID at login
JWT forgery	Trusting the token header's `alg`	Server-side algorithm allowlist + strong keys
Credential stuffing	Password reuse + unlimited attempts	MFA + rate limiting + breached-password checks
Mass assignment	Binding raw request bodies to models	Explicit field allowlists (DTOs)

10.3 Pre-merge security review checklist

☐ Every route declares who may call it (deny by default)	☐ Every object fetch is scoped by user/tenant
☐ All queries parameterized; no raw string SQL	☐ All output contextually encoded / auto-escaped
☐ State-changing endpoints are not GET	☐ CSRF protection on cookie-authenticated routes
☐ Passwords hashed with Argon2id/bcrypt	☐ Session regenerated at login; destroyed at logout
☐ Tokens: short TTL, validated iss/aud/exp/alg	☐ Refresh tokens rotate; reuse detected
☐ Rate limits on auth & expensive endpoints	☐ No secrets in code, logs, or client bundles
☐ Responses use explicit DTOs (no raw rows)	☐ Uniform errors — no user enumeration
☐ Security headers set (helmet + CSP)	☐ Auth events logged; tokens redacted
☐ Dependencies scanned, no known criticals	☐ Input validated with a schema (zod/joi/pydantic)

10.4 One-line mental models

- **Session** = server remembers, client holds a pointer.
- **JWT** = client holds the facts, server holds only a key. Signature = integrity, not secrecy.
- **OAuth2** = "let this app act for me, within these scopes". Not login.
- **OIDC** = OAuth2 that also tells you *who* the user is (id_token).
- **SQLi** = your data became your code.
- **XSS** = their code became your page.

- **CSRF** = their page used your cookie.
- **BOLA** = you checked *who*, never *which*.

11. Practice Labs & Self-Quiz

> **Ground rule:** run every exercise against **your own local application** or a purpose-built practice target (OWASP Juice Shop, DVWA, WebGoat, PortSwigger Web Security Academy labs). Testing systems you don't own or have written permission to test is illegal in most jurisdictions.

11.1 Build labs (do these in order)

Lab 1 — Password + session auth from scratch

1. Express + Postgres/SQLite. Endpoints: `POST /register`, `POST /login`, `GET /me`, `POST /logout`.
2. Hash with Argon2id or bcrypt(12). Store only the digest.
3. Sessions in Redis; cookie `__Host-sid; HttpOnly; Secure; SameSite=Lax`.
4. Implement session regeneration at login and an absolute + idle timeout.
5. **Verify:** capture the sid before and after login — they must differ. Delete the Redis key and confirm the next request returns 401.

Lab 2 — JWT with refresh rotation

1. Issue a 15-minute HS256 access token and a 30-day refresh token (stored *hashed* in the DB with a `family_id`).
2. Middleware verifies with an explicit `algorithms` allowlist and validates `iss` / `aud`.
3. `POST /refresh` rotates: invalidate the old, issue a new pair.
4. Implement reuse detection: replay an old refresh token → the whole family is revoked.
5. **Verify (attack your own code):** (a) tamper with the payload — verification must fail; (b) set `alg` to `none` and strip the signature — must fail; (c) wait past `exp` — must 401; (d) replay a used refresh token — must revoke the family.

Lab 3 — OAuth2 / OIDC login

1. Register an OAuth app with Google or GitHub. Implement Authorization Code + PKCE (S256) with a `state` parameter.
2. Validate the id_token: signature via JWKS, plus `iss`, `aud`, `exp`, `nonce`.
3. Store the account keyed on `iss` + `sub`; require `email_verified`.
4. **Verify:** tamper with the returned `state` → your callback must reject. Reuse an authorization code → the provider must reject.

Lab 4 — Break and fix SQL Injection

1. Write a deliberately vulnerable search endpoint using string concatenation on a throwaway local DB.
2. Confirm the classic bypass works (`' OR '1'='1' --`), then confirm a UNION-style read of another table.
3. Rewrite with parameterized queries; confirm the same payload now returns zero rows.
4. Add a sortable column parameter — implement it with an allowlist map, not interpolation.
5. Add a regression test asserting injection payloads return 0 rows or 400.

Lab 5 — XSS and CSP

1. Build a comment feature that renders with `innerHTML`. Post a comment containing a script tag; confirm it executes (stored XSS).
2. Fix with `textContent`; then add a rich-text variant sanitized with DOMPurify.
3. Add a nonce-based CSP and confirm inline script is now blocked even if injection succeeds.
4. Add a DOM-XSS case reading `location.hash` into a sink, then fix it.

Lab 6 — CSRF

1. Host a second local origin (different port counts as cross-origin, though not cross-*site* — use a distinct hostname via `/etc/hosts` for a real test).
2. From it, auto-submit a POST to your app's `/transfer` endpoint while logged in. Confirm it succeeds.
3. Add `SameSite=Lax` → retest. Add a synchronizer token with constant-time comparison → retest.
4. Add an Origin check that fails closed when the header is missing.

Lab 7 — BOLA hunt

1. Create two users, each with one order. As user A, request user B's order ID. If it returns data, you've found BOLA.

2. Fix by scoping the query with `ownerId`; return 404 rather than 403.
3. Write an automated test for it. Then add rate limiting to your login route and verify 429 after N failures.

11.2 Self-quiz

1. Why is `jwt.decode()` dangerous in an auth middleware?
2. You store a JWT in an HttpOnly cookie. Which attack are you now exposed to that Bearer-header clients are not?
3. What exactly does PKCE prevent, and why do public clients need it?
4. Why must the session ID be regenerated at login?
5. Parameterized queries can't bind an `ORDER BY` column. What do you do instead?
6. Name the three XSS types and which one a server-side WAF cannot see.
7. Why isn't `SameSite=Lax` alone a complete CSRF defense?
8. Give two reasons not to return "email not found" on a failed login.
9. Your access token has a 15-minute TTL. A user is banned. When does their access actually stop, and how do you shorten that?
0. What is the difference between BOLA and broken function level authorization?

Answers

1. `decode()` parses without verifying the signature — anyone can forge any payload. Always use `verify()`.
2. CSRF. Cookies are attached automatically to cross-site requests; the `Authorization` header is not.
3. Authorization-code interception: PKCE binds the code to the client instance that started the flow, so a malicious app that grabs the code can't redeem it without the `code_verifier`. Public clients can't hold a client secret, so PKCE replaces it.
4. Session fixation — otherwise an attacker who planted a known session ID before login keeps a valid authenticated session afterwards.
5. Map user input through a server-side allowlist of permitted column names and directions, with a safe default.
6. Stored, Reflected, DOM-based. DOM-based never reaches the server, so a WAF/server scanner can't see it.
7. Browser support/defaults vary, cross-site **GET** navigations still carry the cookie (so state-changing GETs remain exploitable), and a compromised subdomain is same-site.
8. It enables user enumeration (attacker builds a list of valid accounts for stuffing/phishing), and it usually comes with a timing difference that leaks the same information.
9. Up to 15 minutes later, when the token expires. Shorten it with a `jti` denylist or a `tokenVersion` claim checked against the user record, and by revoking the refresh token immediately.
0. BOLA = allowed to use the endpoint but accessing another user's *object*. Function-level = not allowed to call that *operation* at all (e.g. an admin-only route).

11.3 Where to go next

- **OWASP Top 10** and **OWASP API Security Top 10** — read them once end to end.
- **OWASP Cheat Sheet Series** — the practical reference for every topic in this note.
- **PortSwigger Web Security Academy** — free, hands-on labs for SQLi, XSS, CSRF, OAuth, JWT.
- **RFCs worth skimming:** 6749 (OAuth2), 7636 (PKCE), 7519 (JWT), 8725 (JWT BCP), 9700 (OAuth 2.0 Security BCP), OpenID Connect Core.
- **Habit:** add one security test per feature. Tests are how knowledge survives a refactor.